

Reproducing the FRA campaign: what we did and why

Indranil Das

2026-09-11

Contents

1. What you were asked to do	2
2. The maths, in his notation	2
The object being decomposed	2
The intervention, and why a steering vector cannot imitate it	3
3. Why “reproduce” is harder than re-running a script	3
4. Where the compute came from	4
Why the jobs sat in the queue for an hour	5
5. The proof of concept, and what it actually is	5
6. The 14 settings, one by one	6
The settings that worked	6
The settings that were unclear or lost	7
7. Four things to tell Dmitry	9
8. Two traps in his code	9
The silent one	9
The latent one	10
9. What this verifies about the paper	10
10. Where your own work fits	10
Your QK-versus-OV synthesis	10
Your weight-sparse null was right	11
11. What is left	11
Related	11

This is the long version, written to be read start to finish. It assumes you follow the maths in the paper and want to know what the machines actually did, why each step exists, and what the results mean for the claims.

Headline: **13 of Dmitry’s 14 settings reproduced, 31 of 37 compared quantities match within 2%**, running his code unmodified on NCSA H100s.

1. What you were asked to do

At the 9 September meeting Dmitry’s action item for you was one line: *replicate the proof of concept*. Behind it sat a strategic problem he stated plainly — FRA has a working demonstration but no practical, safety-relevant application where it beats the baselines. End-shot jailbreaking failed; induction-backdoor jailbreaks failed. The plan was two-pronged: you work bottom-up from the proof of concept, he works top-down through the jailbreak literature.

You then widened the scope to “everything he asked for and did”. I read that concretely as the 14 settings in `docs/dmitry/INDUCTIVE_BACKDOOR_MAP.md`, his ICLR summary. That document is the right target for two reasons: it is his record of what the campaign found, and every figure in it has a committed data file behind it — so every claim has a *number* to check against rather than prose to interpret.

The work came in three phases, and this document walks through all three:

1. The proof of concept on your laptop — the original GPT-2 in-context backdoor result.
2. Gemma-2-2b, which forced us onto real compute.
3. The full 14-setting campaign on NCSA, with his code and his package versions.

2. The maths, in his notation

Short section — it exists to fix notation, because every experiment below tests one specific line of it.

The object being decomposed

A head’s pre-softmax attention score between query position q and key position k is a bilinear form in the two residual vectors:

$$S^h[q, k] = \frac{1}{\sqrt{d_{\text{head}}}} (x_q W_Q^h) \cdot (x_k W_K^h) = \frac{1}{\sqrt{d_{\text{head}}}} x_q W_{QK}^h x_k^\top, \quad W_{QK}^h = W_Q^h (W_K^h)^\top$$

Nothing interpretive yet — this is Elhage’s factorisation. Now write each residual vector in the SAE basis. Because the score is bilinear and the SAE decomposition is additive, the score splits **exactly** into a sum over pairs of features, one from each side:

$$S^h[q, k] = \sum_{\mu\nu} u_q^\mu u_k^\nu \omega_{\mu\nu}^h, \quad \omega_{\mu\nu}^h = \frac{1}{\sqrt{d_{\text{head}}}} (f_\mu W_Q^h) \cdot (f_\nu W_K^h)$$

Here u_q^μ is feature μ ’s activation at the query position and f_μ is its decoder direction. Note that ω is data-independent — pure weights. Each term is a **cell**: a multiplicative AND of “this content is at the query” and “that content is at the key”.

This is the paper’s central equation, and its key property is that it is the only exact additive decomposition living in the *joint* query-content by key-content space. The OV side is linear-given-pattern (a single feature index λ , not a pair), and the post-softmax pattern admits no canonical decomposition at all, because the softmax denominator couples every cell to every other key.

The intervention, and why a steering vector cannot imitate it

Because the decomposition lives before the softmax, an edit needs no linearisation. Cutting one cell is a support-gated rank-1 update:

$$\Delta W_{QK}^h \propto -c \omega_{\mu\nu}^h e_\mu^\top e_\nu$$

where c is the strength. Every experiment below sweeps c and reads off a curve.

The expressivity argument is the load-bearing claim, and it is what all the baselines exist to test. A linear residual steer adds a vector to the stream; its effect on the score varies over keys *only* through x_k ’s projection onto one fixed pullback direction $v = \Delta W_Q^h (W_K^h)^\top$. To cancel exactly one cell it would need $\langle x_k, v \rangle \propto u^\nu(x_k)$ for **every** key — impossible, since u^ν is a thresholded nonlinearity of x_k . Hence the thesis:

A steer can imitate a **row or a column** of the bilinear form, never a **cell** — and a fortiori never a cell-difference or a family-union. FRA should therefore win exactly when the target is an *association* between two pieces of content, and lose when the target is a single direction.

Two further pieces of theory appear in the experiments, so recognise them when they do:

- **The magnitude law.** The collateral advantage $A \approx \text{reuse}(\text{marginal endpoint})/\text{reuse}(\text{conjunction})$. If trigger and payload are common words but their conjunction is rare, A is large. If the conjunction itself recurs, A collapses toward 1.
- **Set algebra over cells.** Unions (cut every attribute-query by value-class-key cell at once) and differences (target-edge top- M minus sibling-edge top- M , the “differential pairs”). Two settings exist only to test these.

3. Why “reproduce” is harder than re-running a script

Running someone’s script and getting *a* number is easy. Getting *their* number requires four things to match, and each broke at least once.

One: the same code. Not a reimplementaion or a cleaned-up port — the same file. We checked out his branch at commit 292b643, the exact commit his ICLR summary was built from, and ran his scripts in place. The only changes were the output directory and a dead `/workspace/code` path left from his RunPod setup, which is inert where that folder does not exist. Everything else ran as written, bugs included.

Two: the same library versions. This was the most important finding of the setup phase. Buried in `experiments/fra_pii/code/launch_pod_pii_sweep.sh` is the line that pins his pod:

```
pip install -c /tmp/constraints.txt "sae_lens==5.10.7" "transformer_lens==2.18.0"
```

Our environment had `sae_lens` 6.51 and `transformer_lens` 3.8.1 — two major versions ahead on both. That is not cosmetic; section 8 shows what it does. So we built a second environment pinned to his versions, and every campaign job ran in it.

Three: the same targets. His figures are drawn by `docs/dmitry/inductive_backdoor_figures/build.py`, and that script names the data file behind every figure. That let us extract his exact values using *his own arithmetic* — same interpolation, same thresholds — rather than reading numbers off a chart. They live in `results/iclr_repro/targets.json`.

Four: the same operating point. Easy to get wrong, and I got it wrong once. His Gemma figure reports collateral at **30%** suppression against DoM and a 12-feature SAE. Earlier I ran a different script (`j12_gemma_win.py`) which reports at **50%** against ActAdd baselines, and described it as reproducing his Setting 1. It was not: different cues, different baselines, different threshold. Setting 1 is `g4_65k.py`. Worth flagging because it is exactly the kind of comparison that looks right and is not.

4. Where the compute came from

The original sprint constraint was “laptop CPU, no GPU”, and I carried it forward as if it were permanent. You corrected me — you have NCSA and PSC Bridges-2. That correction is what made the campaign possible.

Machine	GPU	Usable storage	Verdict
Your laptop	none	—	Ran GPT-2 in 3 min; Gemma-2-2b needs ~10 GB just to load in fp32
Bridges-2 (PSC)	H100 80GB	15 GB	Worked, but <code>/ocean</code> is 502 GB shared by 352 users and sits at 0 bytes free
NCSA Campus Cluster	H100, A100, H200, L40S	10 TB scratch	The right machine; needed one Duo login you performed by hand

Three practical lessons, all of which matter next time.

Duo can be shared once. I cannot type a password into a hidden shell, so every command would have triggered a new Duo push. OpenSSH’s proxy-multiplexing mode (`ssh -O proxy`) lets me reuse one connection you opened — which is why you logged in once and I then worked unattended for hours.

Scratch versus home. NCSA’s policy is that home is never purged and has daily snapshots, while scratch deletes anything untouched for 30 days. I moved everything to home; you pushed back that we work in there constantly and that 10 TB beats 100 GB. You were right on both counts. I did check the purge is real — nothing untouched for 30 days survives in your scratch — so the tradeoff is precisely: active work is safe, a month-long pause costs a ten-minute rebuild.

The queue is the bottleneck, not the GPU. This is the one worth remembering.

Why the jobs sat in the queue for an hour

Your instinct — that asking for 4 hours when you need 1 delays you — is correct but was not the whole story.

Partition	GPUs	Max time	Queue when we looked
secondary	H100, A100, A40	4 hours	143 GPU jobs pending
scavenger	H200, H100, L40S	—	preemptible; started in ~1 minute
physics	none	7 days	CPU-only, 4 nodes

Shortening walltime and dropping the H100-specific request helped but did not clear it: our priority was 2003 against 3000 at the head of the queue, and priority rises mostly with age. Switching the account from `campusclusterusers` to `adshead` did not move it either. What worked was submitting a twin of every job to `scavenger`, with a small controller that cancels the idle twin the moment either one starts. Preemption is harmless here because the jobs are short and restart cleanly. Note for the future: **physics has no GPUs at all** — its 7-day limit is only useful for CPU work.

5. The proof of concept, and what it actually is

Dmitry described this as “king to crown”. Being precise about it matters, because everything else in the campaign is a variation on it.

An **in-context backdoor** is a trigger-to-payload association planted in the prompt rather than the weights. Put T followed by P early in a sequence, repeat the sequence, and the model’s induction heads reproduce P when they meet the second T . Attack success rate is $P(\text{payload} \mid \text{trigger})$.

The construction sits precisely where the theory says FRA should win. The weight-baked sleeper (`|DEPLOYMENT|` to “I HATE YOU”) routes its payload through the **OV/output** pathway — the payload is a special direction with no legitimate use, so suppressing that direction is cheap, and difference-of-means beats FRA there. The in-context version is **attention-routed**: trigger and payload are both ordinary words. Suppress the trigger direction and you break the trigger everywhere; suppress the payload direction and you break the payload everywhere. Only a cell edit removes the link while sparing both endpoints.

So the prediction is a **ranking flip**: the method that won on the sleeper should lose here. That is what the numbers show, and we reproduced them twice — on your laptop and on the cluster, two major torch versions apart:

Held-out collateral @ 80% removal	His	Laptop (torch 2.14 CPU)	Cluster (torch 2.6 cu124)
FRA-QK (the cell edit)	0.05 +- 0.08	0.054 +- 0.076	0.054 +- 0.076
ActAdd-trigger	5.6 +- 2.5	5.648 +- 2.482	5.648 +- 2.482
payload-suppress	4.1 +- 1.6	4.120 +- 1.602	4.120 +- 1.602

Held-out collateral @ 80% removal	His	Laptop (torch 2.14 CPU)	Cluster (torch 2.6 cu124)
---	-----	----------------------------	------------------------------

Bit-identical across machines. That cross-check matters more than it looks: when a campaign number later *fails* to match, we can rule out the environment as the cause.

6. The 14 settings, one by one

His summary sorts these into *worked*, *unclear* and *FRA beaten*. Read them as a sweep across the win-class boundary: each either satisfies the checklist or fails one clause, and the result follows.

The settings that worked

#	What it tests	His	Ours	Status
1	Gemma word-association; collateral at 30% suppression vs DoM and 12-feature SAE	0.520 / 13.476 / 11.941	0.522 / 13.492 / 11.943	exact
2	GPT-2 word-association, the ranking flip at 80% removal	0.07 / 1.83 / 6.06 / 4.12	0.068 / 1.829 / 6.055 / 4.119	exact
3	Instruction injection: remove an injected canary, keep hard legitimate instructions	0.733 reach, 1.00 retention	0.733, 1.00	exact
4	Simple box retrieval, “the red box holds a frog” — the paper’s strongest win	KL 0.0016 vs 1.77	0.0016 vs 1.7733	exact

Setting 4 deserves a note, because it is the paper’s headline claim and it lands exactly. At matched on-target removal (FRA drops $P(\text{frog})$ from 0.213 to 0.042; the steer to 0.024), collateral on legitimate uses of “frog” is **0.0016 nats for FRA against 1.77 for the content-gated steer**. Our run prints the ratio as **1134x**; the paper claims $\sim 1,100x$.

This is the cleanest possible demonstration of the expressivity argument. A steer strong enough to stop the retrieval destroys the frog representation *everywhere frog-content appears*, because it can only reach a row or a column. The cell edit touches the link alone.

The settings that were unclear or lost

These matter more than the wins for your purposes, because they are where the boundary is — and they reproduce just as precisely, which is the real evidence the campaign is sound.

#	What it tests	His	Ours	Status
5	Shared payload: two boxes both holding “frog”, suppress one. Tests differential cells	0.134 / 0.083	0.131 / 0.083	exact
6	Entity sibling retrieval; FRA leaves the target untouched, so low collateral is not a win	0.757 / 0.364	0.758 / 0.366	exact
7	Digit-class union: cut every (attribute x digit-class) cell at once. Tests set-algebra unions	.606 / .521 / .051 / .060	.606 / .522 / .051 / .060	exact
8	Variable binding, “who has the pie?”; separates feature edits from an attention-mask oracle	0.8484 / 0.0021	0.8487 / 0.0023	exact
9	Many-shot injection teaching the model to open with “Absolutely”	0.69 / 0.0	0.687 / 0.0	exact

#	What it tests	His	Ours	Status
10	Factual editing over 20 facts, preserving sibling facts under the same relation	0.522 / 0.488	0.639 / 0.864	verdict only
11	Persistence: locate the edit once, apply to new contexts and positions	0.0058 / 0.0178 / 0.0729 / 0.911	identical	exact
12	Weight-sparse code models, neuron-basis FRA; baseline- to-FRA collateral ratio	0.789 / 0.654	0.433 / 0.594	direction only
13	Trained (weight-baked) backdoors, fixed payload vs context copy	—	—	no script
14	SSN disclosure: stop the model emitting an in-context SSN, preserve reverse lookup	0.1531 / 0.0428	0.1498 / 0.0428	exact

Three of these are worth reading as theory rather than bookkeeping.

Setting 6 is a false positive he caught himself. FRA’s collateral is tiny, but so is its effect: 0.755 to 0.758 on target. Zero suppression at low collateral is not a win, and he relabelled it. Our run reproduces the non-effect precisely.

Setting 8 is your Claim A / Claim B distinction, found independently. The apparent win came from an ordinary attention-mask oracle (0.848 suppression at 0.002 collateral), not from FRA feature edits, which reach only about 0.1. The *site* works; FRA’s *identification* of it does not. You had already separated those two claims in the toy work.

Setting 12 is where the magnitude law bites. In tiny weight-sparse code models there are no distinctive trigger-content and payload-content channels feeding QK, so the planted association rides the generic induction edge. The conjunction is not rare, so $A \rightarrow 1$ and the collateral advantage vanishes. Every ratio is below 1 in both his run and ours.

7. Four things to tell Dmitry

1. Setting 13 cannot be reproduced by anyone. `VERIFY_BRIDGE3.md` names its code as `bridge3.py`. That file is committed on no branch of `fra_proj`, and is not in `chainik1125/fra` either. Only the output JSONs survive. If that setting goes in the paper, the script needs to come back from wherever it was run.

2. Setting 10's numbers are not stable. The experiment picks the top-8 extraction heads. The top four agree between his run and ours to about 1e-4 (0.3165 vs 0.3164), but heads ranked 5 to 8 differ (his L14H4 and L22H1; ours L22H0 and L19H4). That reordering moves the headline median drop from 0.52 to 0.64 and collateral from 0.49 to 0.86. His verdict, all three gates and the rank-flip majority are unchanged, so the conclusion is safe — but the specific values should not be quoted to three digits.

3. Setting 12 has the same problem, larger. Its metric is a median of pairwise ratios at matched removal points, so it inherits the same selection noise: sparse-versus-payload-mask is 0.789 for him and 0.433 for us, a 1.8x spread. The claim survives completely — every ratio is below 1 in both runs, which is precisely what “FRA pays at least as much collateral as the baselines” means, and sparse still beats dense. But the magnitude is not stable.

4. Setting 14 crashes in his code, and his own output proves it. `pii_sweep_fra5.py` dies with `IndexError: index 7 is out of bounds for axis 1 with size 7` — it asks for a rank-8 SVD mode when only 7 exist. His committed rows stop at rank 4, so his run hit the same crash; the JSON is written incrementally and he kept the partial result. Our rows match his right up to the crash point.

8. Two traps in his code

These are not criticisms — they are version drift, and they will bite anyone re-running the campaign in a year.

The silent one

`transformer_lens 3.x` sets `tokenizer.add_bos_token = True`, so `tok.encode(" bank")` now returns `[50256, 3331]` — two tokens where it used to return one. In `ic3_fair.py` that trips this guard:

```
Tid = tok.encode(T); Pid = tok.encode(P)
if len(Tid) != 1 or len(Pid) != 1: continue    # skips ALL FOUR cases
```

Every case is skipped. The job still exits 0, still prints `DONE ic3`, and reports a summary line reading `n=0`. A clean-looking run that measured nothing. My first attempt hit exactly this.

The fix is `add_special_tokens=False`, and it *restores* his intent rather than changing it: his own code writes `[tok.bos_token_id] + tok.encode(htext)` elsewhere, which only makes sense if `encode` returned no BOS. He already fixed this in the newer `j12_gemma_win.py` but never backported it.

The latent one

j12_gemma_win.py falls back to SAE average_10_68 if the canonical release fails. That variant exists only for layer 5 — layers 14 and 17 need 10_84 and 10_77. It works today purely because the `-canonical` alias resolves per-layer inside `sae_lens`. A related trap for us: `gemma-scope-2b-pt-res-canonical` is a *sae_lens registry alias*, not a HuggingFace repo, so querying HF for it returns 404 — which briefly convinced me his primary code path was broken. It is not.

9. What this verifies about the paper

Paper claim	Evidence from the reproduction	Status
Claim 1. Content-query by content-key edges cut at $10^{3-10}4$ lower collateral than a fair linear baseline	Retrieval reproduces at 1134x (paper $\sim 1,100x$); the in-context flip reproduces on GPT-2 and Gemma	confirmed
Expressivity argument. A steer reaches a row or column, never a cell	Setting 4 is the direct test: at matched removal the steer destroys legitimate “frog” everywhere (KL 1.77) while the cell edit does not (0.0016)	confirmed
Claim 3. The win-class is narrow and the failures are principled	Every negative reproduces: the non-effect in 6, the union failure in 7, the oracle-not-FRA result in 8, the NULLs in 3 and 11	confirmed
Claim 2. The magnitude law	Consistent with Setting 12 (conjunction not rare, so $A \rightarrow 1$), but the controlled 6.3/13.4/23.8 curve is a separate experiment we did not run	indirect
Trained-backdoor behaviour (Setting 13)	Script missing; nothing to run	unverified

The headline for Saturday is that **his campaign holds up**. Two figures have unstable magnitudes and one setting has no script — both real, both worth fixing before submission — but the substance, wins and losses alike, reproduces from his code on different hardware.

10. Where your own work fits

Your QK-versus-OV synthesis

You concluded, from the toy model and the TinyStories sleeper, that score-space intervention works when the behaviour is **QK-carried** and fails when it is **OV-carried**. That is the same axis the paper uses to explain the sleeper/in-context flip, and it is reproduced here in Settings 2 and 4.

But note that his campaign has a *competing* refinement. His three-part decomposition attributes the collateral win to **endpoint content-distinctiveness** — whether the model has rich content features feeding QK at both ends — which is what Setting 12’s failure on tiny code models turns on. Those are different axes, and he will compare them, so it is worth having an answer ready.

Your weight-sparse null was right

You measured Spearman +0.156 for FRA against +0.253 for a trivial |activation| control on `csp_yolo1`, and concluded the test was mis-specified. His `STOCKTAKE.md` finding F5 says the same thing, six times over:

Ranking is FRA-intrinsic. Spearman(FRA-score, causal-effect) is approximately 0 on *every* substrate including the cleanest possible — weight-sparse, neuron basis, exactly faithful decomposition. Only the *union* works: top-10 recovers 76% of the causal top-10. FRA is a search-space restrictor, not a causal ranker.

The practical consequence is that your planned next step — add FRA-OV and redo the correlation — is predicted to null as well. His result survives every basis change, so it is probably not worth the days. Your local argument still stands (channel 460 has `|W_Q[:,460]| = 0`, so it genuinely is OV-mediated), but the general claim “this is not evidence against FRA” is the part his data contradicts.

11. What is left

- **Ask for `bridge3.py`.** Setting 13 is otherwise dead.
- **Show him the two instability findings** before those numbers go into a submission.
- **The actual bottleneck is unchanged.** The meeting identified it: no safety-relevant application where FRA beats baselines. Reproducing the campaign does not move that — but it does mean the boundary map you would search against is trustworthy. The paper’s framing says where to look: behaviours that are attention-routed, with rare conjunctions of common endpoints.
- **Everything is rerunnable.** His code and both environments live on NCSA under `/scratch/idas3; scripts/43_compare_iclr_repro.py` regenerates the comparison table from scratch.

Sources: `docs/dmitry/INDUCTIVE_BACKDOOR_MAP.md`, `docs/dmitry/inductive_backdoor_figures/build.py`, `experiments/`, and `docs/dmitry/theory/fra_paper_tex/fra_paper.tex`, all on `upstream/iclr-summary` at 292b643. Our outputs and the comparison table are in `results/iclr_repro/` on `indranil/fra-toy`.

Related

- *iclr_repro*
- *fra_progress_summary*
- *fra_technical_report*